

# Structural Reinforcement Learning for Heterogeneous Agent Macroeconomics

Yucheng Yang

Zurich

Joint with Chiyuan Wang, Andreas Schaab, Benjamin Moll

CEF 2026

# Heterogeneous agent models with aggregate risk

---

- Huge literature since Krusell-Smith and Den Haan from late 90s
- Key challenge: rational expectations + general equilibrium  
⇒ distribution = state variable in Bellman equation (“Master equation”)
  - true even though households/firms only care about prices
  - intuition: equilibrium prices are not Markov, only the distribution is  
⇒ forecast distributions to forecast prices

# Heterogeneous agent models with aggregate risk

---

- Huge literature since Krusell-Smith and Den Haan from late 90s
- Key challenge: rational expectations + general equilibrium  
⇒ distribution = state variable in Bellman equation (“Master equation”)
  - true even though households/firms only care about prices
  - intuition: equilibrium prices are not Markov, only the distribution is  
⇒ forecast distributions to forecast prices
- Despite recent impressive advances to solve it directly, still lack of efficient global solution methods for advanced HA models with aggregate risk
- This paper: sidestep master eqn with structural reinforcement learning

## Sidestep Master eqn using structural reinforcement learning

---

RL: learn value & policy functions in Markov decision process with Monte Carlo.

Unlike dynamic programming, RL can handle non-Markov states (e.g. prices).

# Sidestep Master eqn using structural reinforcement learning

---

RL: learn value & policy functions in Markov decision process with Monte Carlo.

Unlike dynamic programming, RL can handle non-Markov states (e.g. prices).

This paper:

- Replace dist'n with low-dim. prices in state space
- RL about equilibrium prices but not individual states  $\Rightarrow$  "Structural RL"
- Efficient asset market clearing using policy functions (= demand curves)
- Structural RL for both households and firms  $\Rightarrow$  solving HANK

# Sidestep Master eqn using structural reinforcement learning

---

**RL**: learn value & policy functions in Markov decision process with Monte Carlo.

Unlike dynamic programming, **RL can handle non-Markov states** (e.g. prices).

This paper:

- **Replace dist'n** with **low-dim. prices** in state space
- **RL about equilibrium prices** but not individual states  $\Rightarrow$  “**Structural RL**”
- Efficient asset market clearing using policy functions (= demand curves)
- **Structural RL** for **both households and firms**  $\Rightarrow$  **solving HANK**

**Outcome**: efficient & flexible global solution method for HA models w agg risk, solves problems **traditional methods struggle with**:

1. non-trivial market clearing (Huggett w agg. risk)  $\approx$  **1 min** on Google Colab
2. portfolio choice  $\approx$  **1 min**
3. HANK with forward-looking price/wage Phillips curve  $\approx$  **4 min**

# Illustrative Model Setup and Methodology

# Textbook HA model – Huggett (1993) with aggregate risk

---

- Continuum of agents  $i$ , heterog. in  $(b_{i,t}, y_{i,t})$ ,  $y_{i,t}$  = id. risk, agg. shock  $z_t$
- Households choose consumption  $c_{i,t}$  to maximize

$$v_{i,0} = \max_{\{c_{i,t}\}} \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t u(c_{i,t}) \quad \text{subject to}$$

$$c_{i,t} + q_t b_{i,t+1} = b_{i,t} + y_{i,t} z_t, \quad y_{i,t+1} \sim \mathcal{T}_y(\cdot | y_{i,t}), \quad z_{t+1} \sim \mathcal{T}_z(\cdot | z_t), \quad b_{i,t+1} \geq \underline{b}$$

- State of the economy: distribution  $G_t(b, y)$  and agg. shock  $z_t$
- Market clearing: bond price  $q_t$  such that

$$\int b'_t(b, y) dG_t(b, y) = 0, \quad \text{all } t$$

**Note:** agent problem depends on  $G_t$  only through low-dimensional price  $q_t$



## Key difficulty: equilibrium prices are not Markov

---

- Households care about price  $q_t$ , not directly about distribution  $G_t$
- But low-dim equilibrium prices  $q_t = P^*(G_t, z_t)$  is not first-order Markov
- ... only extremely high-dimensional  $(G_t, z_t)$  is
- Dynamic programming can only handle Markov states  $\Rightarrow$  Master equation

$$V(b, y, G, z) = \max_{c, b'} \{ u(c) + \beta \mathbb{E}[V(b', y', G', z') \mid b, y, G, z] \}$$

subject to  $c + q(G, z)b' = b + yz, b' \geq \underline{b}$ .

# Structural RL to sidestep the master equation

---

- Our solution: use **policy gradient** to solve for low-dimensional policy

$$c_{\theta}(b, y, \mathbf{q}, z)$$

to maximize expected lifetime utility on **simulated equilibrium paths**

$$v(\theta) = \mathbb{E}_{(b_0, y_0, G_0, z_0)} \left[ \sum_{t=0}^T \beta^t u(c_{\theta}(b_t, y_t, \mathbf{q}_t, z_t)) \right]$$

subject to  $c + q(G, z)b' = b + yz, b' \geq \underline{b}$ . ► Restricted perceptions equilibrium

# Structural RL to sidestep the master equation

- Our solution: use **policy gradient** to solve for low-dimensional policy

$$c_{\theta}(b, y, \mathbf{q}, z)$$

to maximize expected lifetime utility on **simulated equilibrium paths**

$$v(\theta) = \mathbb{E}_{(b_0, y_0, G_0, z_0)} \left[ \sum_{t=0}^T \beta^t u(c_{\theta}(b_t, y_t, \mathbf{q}_t, z_t)) \right]$$

subject to  $c + q(G, z)b' = b + yz, b' \geq \underline{b}$ . ► Restricted perceptions equilibrium

- **Assumption 1:** agents observe **prices**  $\mathbf{q}_t$  but not full **distribution**  $G_t(b, y)$
- **Assumption 2:** policy depends only on **current** prices  $\pi(b, y, \mathbf{q}, z)$ .

Companion paper (ICML '26): extend to price histories via **Recurrent SRL**

**Similarity** to standard RL: don't know transition prob. of prices  $p_t$

**Difference** to standard RL: **know “local environment”**  $\mathcal{T}_s$  and  $u$  (Han-Yang-E)

## Simulating the economy given policy $c_\theta(b, y, q, z)$

---

Given policy  $c_\theta(b, y, R, z)$  and  $(G_0, z_0)$ , want to simulate economy forward:

$$\{G_t, q_t, z_t\}_{t \geq 0}$$

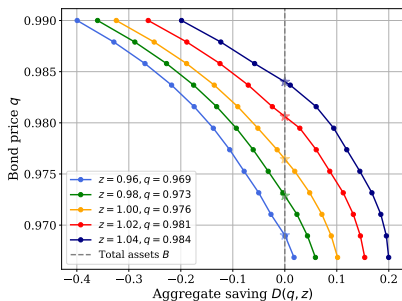
- $z_{t+1}$  is easy: exogenous aggregate shock process
- $G_{t+1}$  is easy when given  $q_t, z_t, G_t$ , and policy
- $q_t$  is hard: pinned down implicitly by market clearing each period

# Efficient handling of non-trivial market clearing

$$D_t(\mathbf{q}, z) = \bar{B} = 0, \quad D_t(\mathbf{q}, z) := \int b'(b, y, \mathbf{q}, z) dG_t(b, y) = \text{agg. saving supply}$$

Key: integrate policy  $b'(b, y, \mathbf{q}, z) \Rightarrow$  aggregate saving on price grid  $S(\mathbf{q}, z_t)$

$$\Rightarrow q_t \text{ solves } S_t(\mathbf{q}_t, z_t) = \mathbf{b}'(\mathbf{q}_t, z_t)^\top \mathbf{g}_t = 0$$



Market clearing is part of environment, not another loop!

# Structural RL Summary

---

- Parameterize a low-dimensional household policy

$$c_{\theta}(b, y, \mathbf{q}, z)$$

- Choose  $\theta$  to maximize lifetime utility over simulated equilibrium rollouts:

$$v(\theta) \approx \hat{v}(\theta) = \frac{1}{N} \sum_{n=1}^N \sum_{t=0}^T \beta^t u(c_{\theta}(b_t^n, y_t^n, \mathbf{q}_t^n, z_t^n))$$

- No perceived law of motion; no inner loop / outer loop as in Krusell-Smith
- Optimize  $\theta$  by policy gradient with structural knowledge on  $b, y$  dynamics and  $u(\cdot)$  (detailed discussion in the paper)
  - low-dimensional state  $\Rightarrow$  grid-based  $\pi_{\theta}$  works well in practice

Computational experiments

# Runtimes

---

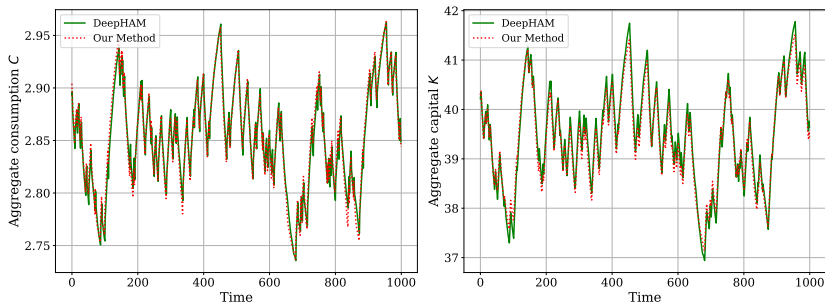
- Efficient implementation in JAX for GPUs, run on Google Colab
- Stochastic algorithm: present averages over multiple runs

| Model                         | Average converge epoch | # Runs | Average Runtime (sec) |
|-------------------------------|------------------------|--------|-----------------------|
| Krusell-Smith                 | 462.3                  | 10     | 36.77                 |
| Huggett with agg. shocks      | 573.0                  | 10     | 45.91                 |
| Portfolio choice              | 637.5                  | 10     | 84.3                  |
| HANK with agg. shocks         | 707.5                  | 10     | 246.49                |
| Partial Equilibrium (Huggett) | 355.8                  | 10     | 24.50                 |

Note: all experiments were implemented on the A100 GPU on Google Colab

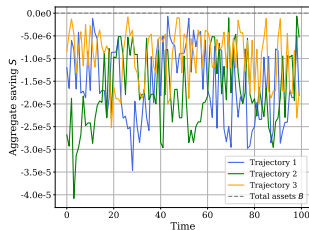
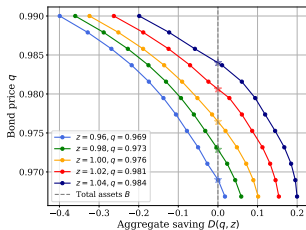
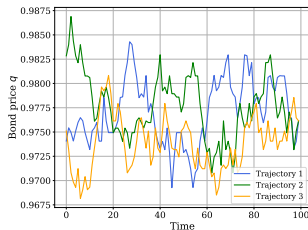
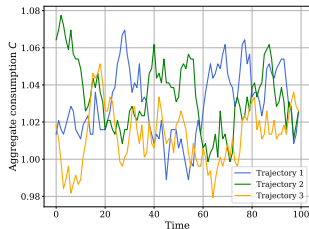
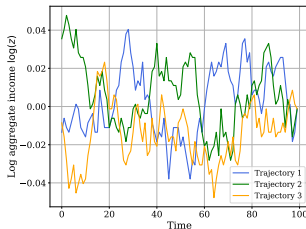
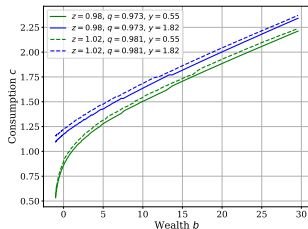


# KS model: Structural RL method recovers RE solutions

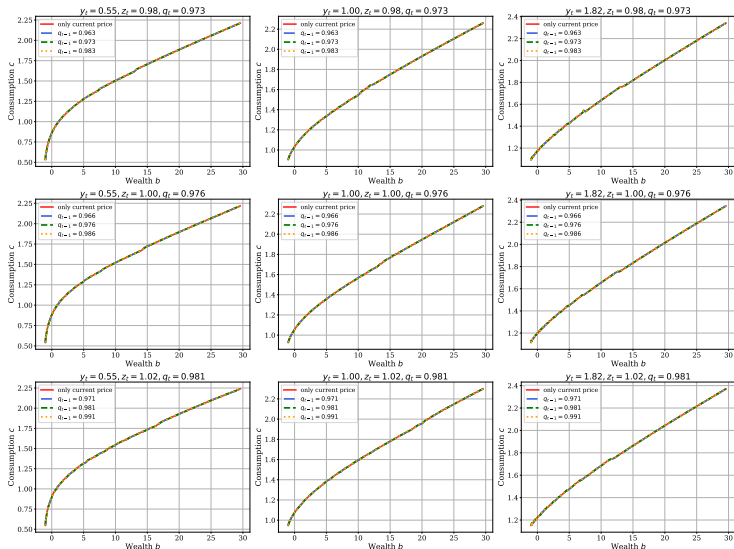


RE solutions are obtained with a deep learning based method (DeepHAM).

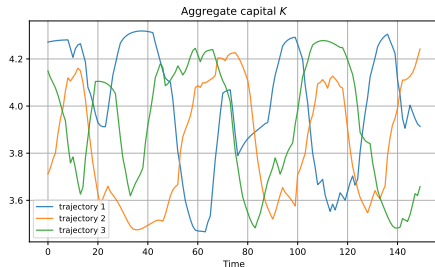
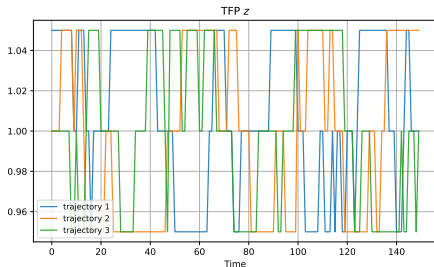
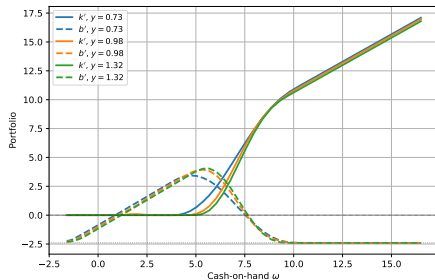
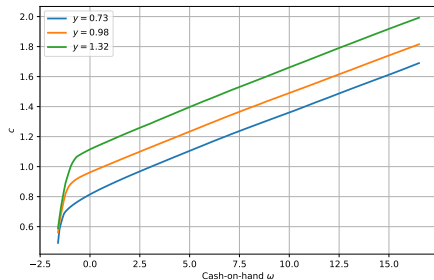
# Huggett: Simulated trajectories under optimal policy



# Adding one lagged price $p_{t-1}$ into state space



# Portfolio choice: KS 1997



# HANK: policy gradient method for both households & firms

---

**Household block** (similar to before): states  $s = (b, y)$  and prices  $p = (q, w)$

policies =  $\{c(s, p), n(s, p)\}$  that maximize PDV of utility

**Firm block**: price setting  $\Rightarrow$  forward-looking Phillips curve = added difficulty

$$\Pi_t = \frac{\varepsilon}{\theta} \left( \frac{w_t}{z_t} - m^* \right) + \mathbb{E} \left[ \Lambda_{t \rightarrow t+1} \frac{Y_{t+1}}{Y_t} \Pi_{t+1} \middle| \mathcal{I}_t \right], \quad m^* = \frac{\varepsilon - 1}{\varepsilon}$$

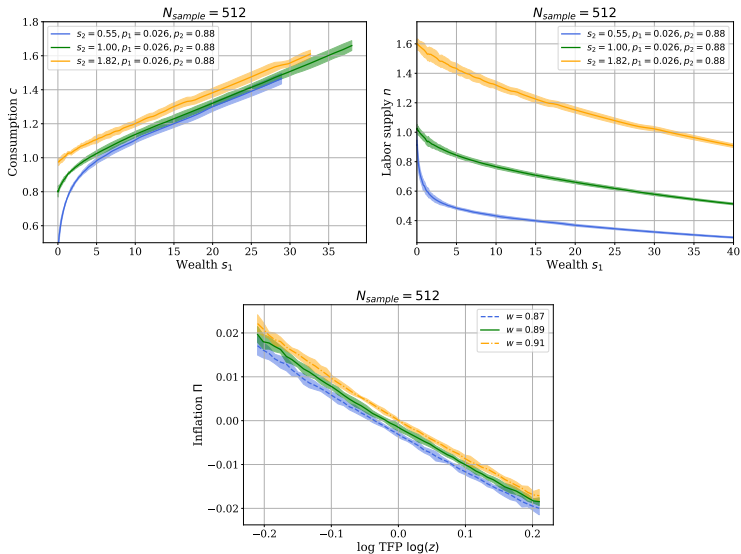
Conventional approach: parameterize  $\mathbb{E}[\Pi_{t+1} | \mathcal{I}_t] \Rightarrow$  complicated fixed-point (e.g. Kase-Melosi-Rottner)

Our solution: solve firm price-setting problem using policy gradient method

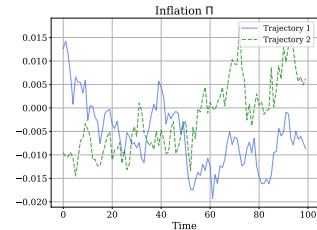
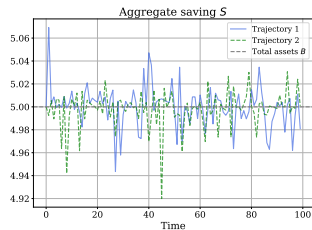
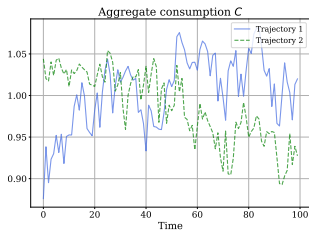
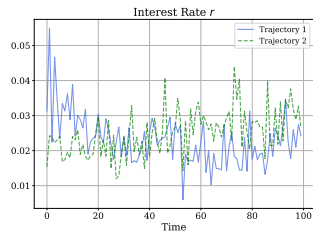
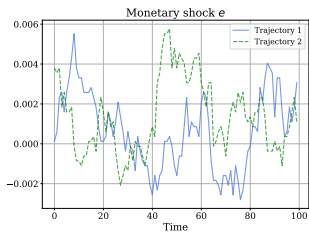
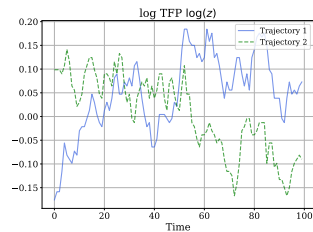
**Symmetric treatment** of firms and households, update policies simultaneously

In practice: good convergence properties

# HANK: Household and firm policy functions



# HANK simulations



# Summary

---

Efficient and flexible **global** solution method for non-stationary HA models

**“Structural RL”**: hybrid RL method that exploits known model structure

- RL about eqm prices but not individual states
- sidestep infinite-dimensional Master equation
- solve much lower-dimensional problem

Solves problems traditional methods struggle with

- non-trivial market clearing conditions
- HANK with forward-looking Phillips curve



# Summary

---

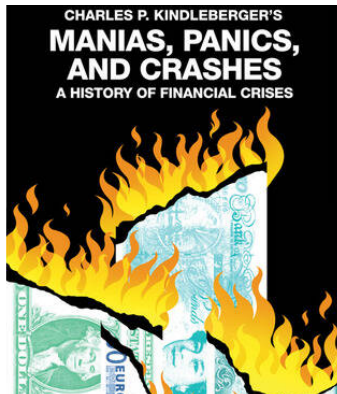
Efficient and flexible **global** solution method for non-stationary HA models

“**Structural RL**”: hybrid RL method that exploits known model structure

- RL about eqm prices but not individual states
- sidestep infinite-dimensional Master equation
- solve much lower-dimensional problem

Solves problems traditional methods struggle with

- non-trivial market clearing conditions
- HANK with forward-looking Phillips curve
- next: models of **large crises**, booms/busts



Thanks!

In stationary world, lagged prices are enough for RE [▶ back](#)

---

Recall **Assumption 1**: agents observe prices  $p_t$  but not distribution  $G_t(s)$

Important: Assumption 1 still consistent with rational expectations

Why? Wold representation theorem!

Step 1 (Wold): if  $p_t$ -process is stationary, it has Wold representation = VMA( $\infty$ )

$$p_t = \sum_{j=0}^{\infty} c_j \varepsilon_{t-j}, \quad c_j = \text{some unknown coefficients}$$

Step 2: if VMA( $\infty$ ) is invertible, it can be expressed as a VAR( $\infty$ ) and hence

$$p_{t+1} \sim \mathcal{T}_p(\cdot | p_t, p_{t-1}, \dots)$$

In practice, include finitely many lags

**Assumption 2**: extreme case with zero lags  $p_{t+1} \sim \mathcal{T}_p(\cdot | p_t)$

# Restricted perceptions equilibrium ► back

---

A pair of mappings  $(\pi^*, P^*)$  constitutes a restricted perceptions equilibrium if:

1. Optimality. For any price sequence  $\{p_t\}$  generated by  $p_t = P^*(\mathbf{g}_t, z_t)$  and exogenous sequence  $\{z_t\}$ , agents choose  $\pi^*(s, p, z)$  to solve:

$$\max_{\pi} \mathbb{E} \left[ \sum_{t=0}^{\infty} \beta^t u(\pi(s_t, p_t, z_t)) \right],$$

subject to the individual budget constraint and state transition equations.

2. Market clearing. For every period  $t$ , all markets clear.
3. Consistency. The prices that agents use to form expectations coincide with the prices in the simulated economy when all agents follow  $\pi^*$ :

$$p_t = P^*(\mathbf{g}_t, z_t), \quad \mathbf{g}_{t+1} = \mathbf{A}_{\pi^*(p_t, z_t)}^{\top} \mathbf{g}_t$$

# Key difficulty: equilibrium prices are not Markov

---

- Equilibrium prices satisfy [▶ back](#)

$$p_t = P^*(\mathbf{g}_t, z_t)$$

$$\mathbf{g}_{t+1} = \mathbf{A}_{\pi_t(z_t)}^\top \mathbf{g}_t$$

$$z_{t+1} \sim \mathcal{T}_z(\cdot | z_t)$$

- Difficulty: low-dimensional  $p_t$  does not have Markov property...
- ... only extremely high-dimensional  $(\mathbf{g}_t, z_t)$  does
- Dynamic programming can only handle Markov states  $\Rightarrow$  Master equation
$$V(s, \mathbf{g}, z) = \max_c u(c) + \beta \mathbb{E} [V(s', \mathbf{g}', z') | s, \mathbf{g}, z] \quad \text{s.t. } s' \sim \mathcal{T}_s(\cdot | s, c, P^*(\mathbf{g}, z))$$
- Without Markov transition prob's: cannot even write Bellman equation!
- But what if there was a way to approximate value and policy functions with  $p_t$  process for which there are no Markov transition probabilities?

# Brief primer on reinforcement learning

RL: learning value & policy functions in incompletely-known Markov decision processes from experience (Monte Carlo sampling) a.k.a. “approximate DP”



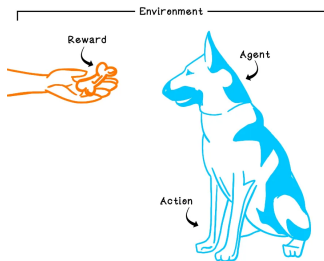
## Playing Atari with Deep Reinforcement Learning

Volodymyr Mnih Koray Kavukcuoglu David Silver Alex Graves Ioannis Antonoglou

Daan Wierstra Martin Riedmiller

DeepMind Technologies

{vlad,koray,david,alex.graves,ioannis,daan,martin.riedmiller} @ deepmind.com



# Computing an expected value

---

Random variable  $x$

How compute expected value  $\mathbb{E}[x]$ ? Two approaches:

1. **Exact:** know probability distribution  $f(x) \Rightarrow$  calculate

$$\mathbb{E}[x] = \int x f(x) dx$$

2. **Monte Carlo:** don't know  $f$  but can sample  $\{x_1, x_2, \dots, x_N\}$

$$\mathbb{E}[x] \approx \bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$



# Computing an expected value

---

Random variable  $x$

How compute expected value  $\mathbb{E}[x]$ ? Two approaches:

1. **Exact:** know probability distribution  $f(x) \Rightarrow$  calculate

$$\mathbb{E}[x] = \int x f(x) dx$$

2. **Monte Carlo:** don't know  $f$  but can sample  $\{x_1, x_2, \dots, x_N\}$

$$\mathbb{E}[x] \approx \bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

Or update incrementally (stochastic approximation method):

$$\bar{x}_k = \frac{1}{k} \sum_{i=1}^k x_i \quad \text{satisfies} \quad \bar{x}_k = \bar{x}_{k-1} + \frac{1}{k} (x_k - \bar{x}_{k-1}), \quad \frac{1}{k} = \text{“learning rate”}$$

# Computing a value function

---

For now: eliminate actions and individual states

$$v_0 = \mathbb{E} \left[ \sum_{t=0}^{\infty} \beta^t u(p_t) \right], \quad p_t = \text{exogenous stochastic process}$$

Two approaches:

1. **Dynamic programming:**  $p_t$  Markov and know  $f(p'|p)$

$$v(p) = u(p) + \int v(p') f(p'|p) dp'$$

# Computing a value function

---

For now: eliminate actions and individual states

$$v_0 = \mathbb{E} \left[ \sum_{t=0}^{\infty} \beta^t u(p_t) \right], \quad p_t = \text{exogenous stochastic process}$$

Two approaches:

1. **Dynamic programming:**  $p_t$  Markov and know  $f(p'|p)$

$$v(p) = u(p) + \int v(p') f(p'|p) dp'$$

2. **Monte Carlo:** don't know  $f$  but sample  $N$  trajectories  $\{p_t^i\}_{t=0}^T$

$$v_0 \approx \hat{v}_0 = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \beta^t u(p_t^i) \quad \text{or} \quad \hat{v}_0^k = \hat{v}_0^{k-1} + \frac{1}{k} \left( \sum_{t=0}^T \beta^t u(p_t^k) - \hat{v}_0^{k-1} \right)$$

# Computing a value function

---

For now: eliminate actions and individual states

$$v_0 = \mathbb{E} \left[ \sum_{t=0}^{\infty} \beta^t u(p_t) \right], \quad p_t = \text{exogenous stochastic process}$$

Two approaches:

1. **Dynamic programming:**  $p_t$  Markov and know  $f(p'|p)$

$$v(p) = u(p) + \int v(p') f(p'|p) dp'$$

2. **Monte Carlo:** don't know  $f$  but sample  $N$  trajectories  $\{p_t^i\}_{t=0}^T$

$$v_0 \approx \hat{v}_0 = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \beta^t u(p_t^i) \quad \text{or} \quad \hat{v}_0^k = \hat{v}_0^{k-1} + \frac{1}{k} \left( \sum_{t=0}^T \beta^t u(p_t^k) - \hat{v}_0^{k-1} \right)$$

Can also be extended to compute optimal policy: **policy gradient method**

# Computing a value function

---

For now: eliminate actions and individual states

$$v_0 = \mathbb{E} \left[ \sum_{t=0}^{\infty} \beta^t u(p_t) \right], \quad p_t = \text{exogenous stochastic process}$$

Two approaches:

1. **Dynamic programming:**  $p_t$  Markov and know  $f(p'|p)$

$$v(p) = u(p) + \int v(p') f(p'|p) dp'$$

2. **Temporal difference learning:**  $N$  trajectories, update  $v$  incrementally

$$\hat{v}^k(p_t^k) = \hat{v}^{k-1}(p_t^k) + \frac{1}{k} \left( \left[ \sum_{\tau=0}^{n-1} \beta^\tau u(p_{t+\tau}^k) + \beta^n \hat{v}^{k-1}(p_{t+n}^k) \right] - \hat{v}^{k-1}(p_t^k) \right)$$

Can also be extended to compute optimal policy: **policy gradient method**

# Agent vs Environment, Rollout of a Policy

---

RL distinguishes between **agent** and **environment** = everything outside of agent

In HA macro:

- **agents** = households and their policies
- **environment = everything else**, including market clearing etc

Related: **rollout** = agent interacting with environment **under given policy**

- take any (generally suboptimal) policy, run it forward in time
- how optimize policy is completely **separate question** (policy improvement)

This viewpoint will be important, in particular for non-trivial market clearing